

CC3_Comp Manual

CC3_Comp is the PC-side compressor and packer for the CoCo 3 SuperLoader. It reads one or more CoCo LOADM files, simulates the CoCo 3 MMU while those LOADM records are applied, builds a 2 MB physical RAM map, and then writes a stream of compressed packets for the 6809 loader.

The matching 6809 side is intentionally simple and fast. All expensive work, such as range ordering, match searching, and compression experiments, happens on the PC.

Quick Start

Most users should start with the project makefile:

```
make
```

To name the project and assemble a different source file:

```
make PROJECT=GALAGA PROGRAM_ASM=Galaga_12.asm
```

To pass compressor options through the makefile:

```
make COMP_OPTIONS="-M78 -Z00"
```

You can also run the compressor directly from the loader folder:

```
./CC3_Comp NEW.BIN -oGO-V12
./CC3_Comp NEW.BIN -oGO-V12 -M78 -Z00
```

When running it directly, run it from the folder that contains MOVER_TEMPLATE.BIN and CC3X0.BIN, or keep those files in the current working directory. The template makefile handles this by changing into CC3_SuperLoader before launching CC3_Comp.

If you are building CC3_Comp from CC3_Comp.bas yourself with QB64PE:

```
./qb64pe -x CC3_Comp.bas -o CC3_Comp
```

What The Program Does

CC3_Comp performs these main steps:

1. Reads a single LOADM .BIN file, or a .LST file that names several LOADM files.
2. Simulates the CoCo 3 MMU registers while LOADM records are loaded.
3. Builds the final 2 MB physical RAM image that the program expects.
4. Tracks exactly which bytes were actually written by the user's program.
5. Reserves loader, Disk BASIC shadow, and source-window blocks.
6. Optionally fills internal gaps inside used MMU blocks with -Zxx.
7. Builds used memory ranges and optionally reorders them with -M methods.
8. Compresses the ranges into the CC3X0 packet stream.
9. Splits the stream into COMP####.BIN files that Disk BASIC can load.
10. Patches MOVER.BIN for the chosen loader/shadow block layout.

11. Writes a report showing the final block map and compression details.

The compressor does not shell out to a separate ZX0 tool and does not require helper Python programs.

Input Files

Single LOADM File

```
./CC3_Comp NEW.BIN -oG0-V12
```

NEW.BIN must be a CoCo LOADM-format binary.

Multiple LOADM Files

```
./CC3_Comp FILES.LST -oG0-V12
```

FILES.LST is a text file with one LOADM file per line. Blank lines and lines beginning with *, ;, or # are ignored.

Example:

```
MAIN.BIN
SPRITES.BIN
TABLES.BIN
```

Optional Loading Screens

You can add a single text-screen CSV:

```
./CC3_Comp NEW.BIN -oG0-V12 TITLE.CSV
```

Or a .SCNS file that changes the loading screen at percentage points:

```
./CC3_Comp NEW.BIN -oG0-V12 LOADSCREENS.SCNS
```

Example .SCNS file:

```
0,title.csv
50,halfway.csv
85,almostdone.csv
```

Each CSV is parsed as byte values from 0 to 255. The loader uses the top 480 bytes of the 32-column text screen for the background and keeps the last row for the loading progress display.

Output Files

CC3_Comp writes these files:

File	Purpose
OUTNAME.CC3X0	Complete compressed packet stream.
OUTNAME_Report.txt	Human-readable report with block map, packet info, and compression totals.
MOVER.BIN	Patched first-stage mover loaded and executed by Disk BASIC.
COMP####.BIN	8K-or-smaller stream chunks loaded one at a time by Disk BASIC.

STREAMFILES.LST	List of generated COMP####.BIN files.
DISKFILES.LST	Disk number and file assignment for each COMP####.BIN.

The makefile also uses:

File	Purpose
START.BAS	Disk BASIC program that loads the mover, decompressor, and stream chunks.
CC3X0.BIN	6809 streaming decompressor loaded at \$0F00.
MOVER_TEMPLATE.BIN	Generic mover template that CC3_Comp patches into MOVER.BIN.

Runtime Loading Flow

The BASIC loader on DISK1.DSK does the high-level disk I/O. That keeps the loader compatible with the normal CoCo Disk BASIC disk routines and disk controllers.

The normal flow is:

1. START.BAS loads and executes MOVER.BIN.
2. MOVER.BIN protects the RAM that BASIC still needs, prepares the stack, and

returns to BASIC.

3. START.BAS loads CC3X0.BIN, the streaming decompressor.
4. BASIC loads COMP0000.BIN into the source window at \$2000-\$3FFF.
5. BASIC executes the decompressor at \$0F00.
6. The decompressor consumes the current chunk and updates a status byte.
7. BASIC either loads the next chunk, prompts for the next disk, or stops

because the program has started.

The decompressor status byte is at \$0F03:

Value	Meaning
0	Load the next COMP####.BIN.
1	Ask the user to insert the next disk before continuing.
255	Stream error detected.

When the final packet is decoded, the decompressor restores the final MMU state and jumps to the user's program.

Command Line Reference

```
CC3_Comp FILENAME.BIN [-oOUTNAME] [-v] [-q] [-M#|-A...] [-Zxx] [-max####] [-chain####] [SCREEN.CSV|SCREENS.SCNS]
CC3_Comp FILES.LST [-oOUTNAME] [-v] [-q] [-M#|-A...] [-Zxx] [-max####] [-chain####] [SCREEN.CSV|SCREENS.SCNS]
```

Option	Meaning
--------	---------

-h, --help, -?	Show built-in help.
-oOUTNAME	Set output base name. Writes OUTNAME.CC3X0 and OUTNAME_Report.txt.
-v	Verbose output. Shows progress and packet details.
-v2	Extra verbose LOADM parsing detail.
-q	Quick mode. Disables one-byte lazy matching.
-A	Auto-select from the balanced compression method set.
-Afast	Auto-select from a quick candidate set.
-Abest	Auto-select from a slower candidate set.
-Aall	Auto-select from a very slow experimental candidate set.
-M#	Select compression method. Default is -M0.
-Zxx	Fill internal unused gaps inside used MMU blocks with hex byte xx.
-Z	Same as -Z00.
-max####	Maximum uncompressed bytes per packet, 256 to 8192. Default is 8192.
-chain####	Maximum hash-chain matches tested per lookup, 1 to 4096. Default is 2048.
-checks####	Alias for -chain####.

By default the program is quiet. Use -v when you want a detailed run log.

Auto Compression

The -A options make CC3_Comp try several -M methods, pick the best candidate, then rebuild the final stream with the winning method.

Auto mode does not try several -Z values. If you supply -Z, the same zero-fill setting is used for every auto candidate. If you do not supply -Z, auto mode leaves zero-fill off.

Tie-breaking is intentionally practical:

1. Prefer a candidate that is more than 16 bytes smaller.
2. If candidates are within 16 bytes, prefer the faster one.
3. If still tied, keep the earlier/simpler candidate.

Auto mode results are written to OUTNAME_Report.txt, with the winner marked by *. Candidates are tested in method-family order from -M0 through -M9.

Option	Candidate methods
-Afast	-M0, -M1, -M4, -M78, -M832
-A	-M0, -M1, -M2, -M3, -M4, -M5, -M6, -M78, -M716, -M832, -M95
-Abest	-A plus -M732, -M816, -M864, -M98
-Aall	-Abest plus extra -M7, -M8, and -M9 hash lengths in method order

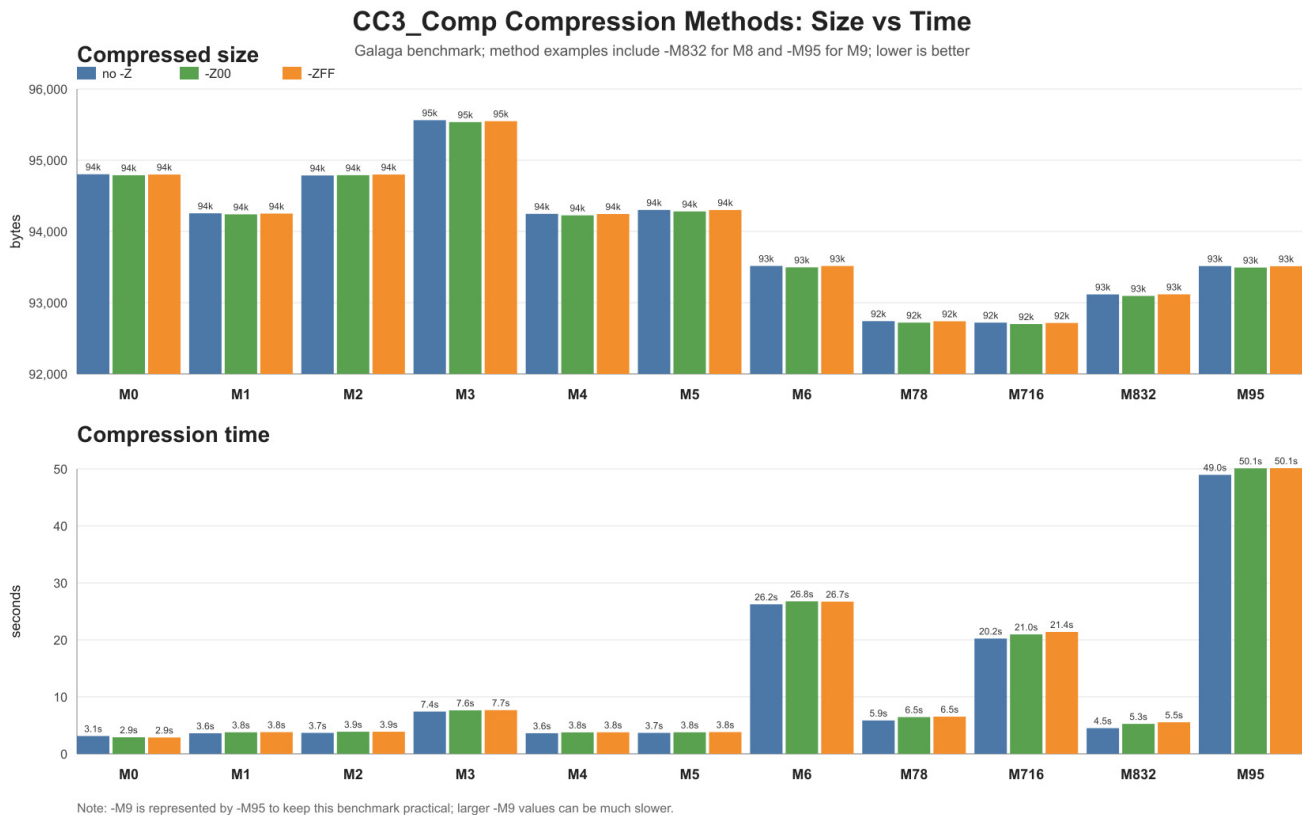
Examples:

```
./CC3_Comp NEW.BIN -oGO-V12 -A
./CC3_Comp NEW.BIN -oGO-V12 -A -Z00
./CC3_Comp NEW.BIN -oGO-V12 -Afast
./CC3_Comp NEW.BIN -oGO-V12 -Abest
```

Compression Methods And Gap Fill

The -M and -Z options change only the compressor. They do not require a different 6809 decompressor and do not change the packet format.

Because the 6809 decoder and packet commands stay the same, decompression time usually does not change much from one -M method to another. The advanced methods mostly spend more time on the PC to choose a smaller command stream. That can reduce disk loading time because fewer compressed bytes need to be read, but the CPU-side decompression loop is still doing the same basic literal and copy operations.



The graph above was generated from the Galaga build used during loader testing. Results will vary by program, but it gives a useful starting point. The parameterized methods use representative settings: -M78 and -M716 for method 7, -M832 for method 8, and -M95 for method 9. Larger -M9 values can take much longer.

-M Methods

Method	Description
-M0	Original packet order. This is the default and is the safest baseline.

-M1	Range/block ordering before compression. Decodes high-value reusable blocks earlier so later packets can copy from them.
-M2	Splits large ranges into smaller reorderable pieces, then applies range/block ordering.
-M3	Range/block ordering plus bounded optimal/lazy parsing. Tries more parse choices than the normal one-byte lazy matcher.
-M4	Range/block ordering plus repeated-fill detection. Encodes byte fills using normal overlapping copy commands.
-M5	Repeated-fill detection plus small repeating-pattern detection.
-M6	-M5 plus wider PC-side 3-byte and 4-byte hash match searching.
-M7###	-M6 plus a user-selected long hash length from 4 to 999 bytes.
-M8###	-M7 style searching, but tries long hash lengths from ### down to 4 bytes.
-M9###	Exhaustive search that tries the broad 4-byte hash first, then 5 bytes up through ###.

-M7### is parsed as method 7 plus the hash length. For example:

```
./CC3_Comp NEW.BIN -oGO-V12 -M78
./CC3_Comp NEW.BIN -oGO-V12 -M716
```

-M78 means method 7 with an 8-byte hash. -M716 means method 7 with a 16-byte hash.

-M8### is parsed the same way, but it searches more hash lengths. For example:

```
./CC3_Comp NEW.BIN -oGO-V12 -M816
```

-M816 means method 8 with descending long-hash searches from 16 bytes, then 15, then 14, and so on down to 4. This can find matches that a single -M7 hash length misses, but it uses more RAM and can take much longer. The 6809 decompressor and packet format are unchanged.

-M9### is the slower exhaustive version. For example:

```
./CC3_Comp NEW.BIN -oGO-V12 -M916
```

-M916 searches the 4-byte hash first, then 5, 6, 7, and so on up to 16, keeping the best match found. This is useful when you want to see whether the broad 4-byte chain can find a better match than the long-hash-only path. It can be much slower than -M8, and it may not produce a smaller file, so treat it as a release-build experiment.

-Zxx Internal Gap Fill

By default, CC3_Comp compresses only bytes that the user's program actually wrote. Unused bytes are not touched by the loader.

-Zxx changes that only for internal gaps inside an already used 8K MMU block. It finds the first used byte and the last used byte in each used block. Any unused byte between those two points is filled with the requested hex value and becomes part of the compressed stream.

Example inside one 8K MMU block:

```
Used range 1: $0500-$0800
Unused gap:  $0801-$0FFF
Used range 2: $1000-$1200
```

With -Z00, the compressor fills \$0801-\$0FFF with \$00. It does not fill \$0000-\$04FF, and it does not fill \$1201-\$1FFF, because those are outside the first-to-last used span.

Valid forms:

```
./CC3_Comp NEW.BIN -oGO-V12 -Z
./CC3_Comp NEW.BIN -oGO-V12 -Z00
./CC3_Comp NEW.BIN -oGO-V12 -Zff
```

-Z without a value means -Z00. Hex case does not matter.

Use -Z when those internal holes are safe to define. It can improve compression by creating repeatable bytes that later ranges can copy from. Do not use it if your program intentionally depends on those internal unused bytes being untouched or random.

Tested Benchmark Data

This benchmark used 8K packets, lazy matching enabled, and chain 2048 unless a method internally did more searching. The original Galaga LOADM input file used for this test was NEW.BIN, 230,950 bytes. Of that, 230,715 bytes were actual program bytes touched by LOADM records and MMU writes. The table shows the final compressed packet-stream size before LOADM chunk overhead.

Method	No -Z	-Z00	-ZFF
-M0	94,803 bytes, 3.13s	94,789 bytes, 2.91s	94,799 bytes, 2.87s
-M1	94,255 bytes, 3.61s	94,240 bytes, 3.78s	94,251 bytes, 3.81s
-M2	94,786 bytes, 3.68s	94,790 bytes, 3.88s	94,799 bytes, 3.88s
-M3	95,562 bytes, 7.42s	95,535 bytes, 7.63s	95,548 bytes, 7.67s
-M4	94,247 bytes, 3.62s	94,226 bytes, 3.77s	94,245 bytes, 3.78s
-M5	94,302 bytes, 3.68s	94,281 bytes, 3.78s	94,301 bytes, 3.82s
-M6	93,516 bytes, 26.24s	93,496 bytes, 26.76s	93,515 bytes, 26.71s
-M78	92,741 bytes, 5.86s	92,720 bytes, 6.45s	92,739 bytes, 6.53s
-M716	92,720 bytes, 20.23s	92,700 bytes, 20.96s	92,715 bytes, 21.39s
-M832	93,116 bytes, 4.51s	93,095 bytes, 5.26s	93,117 bytes, 5.54s
-M95	93,514 bytes, 48.96s	93,493 bytes, 50.10s	93,512 bytes, 50.13s

For this build, the smallest tested output was:

```
./CC3_Comp NEW.BIN -oGO-V12 -M716 -Z00
```

Example result for the 230,950-byte Galaga source: 92,700 compressed packet bytes before LOADM chunk overhead.

The best fast result was:

```
./CC3_Comp NEW.BIN -oGO-V12 -M78 -Z00
```

Example result for the same 230,950-byte source: 92,720 compressed packet bytes.

The fastest strong optimizer in this benchmark was:

```
./CC3_Comp NEW.BIN -oGO-V12 -M832 -Z00
```

Example result for the same 230,950-byte source: 93,095 compressed packet bytes.

Those are good experiments after first proving your program loads with the default -M0.

Recommended Workflow

Start with the raw compressor default:

```
make COMP_OPTIONS=
```

That clears makefile-supplied options and uses the compressor's raw -M0 behavior. Verify that the program loads and runs correctly. Then try:

```
make COMP_OPTIONS="-M78"
make COMP_OPTIONS="-M78 -Z00"
make COMP_OPTIONS="-M716 -Z00"
```

Compare the generated OUTNAME_Report.txt files and test the resulting disk images. Smaller compressed output is useful only if the program still starts and runs correctly.

For quick development builds, use the default -M0 or a fast method. For a release build, try the slower methods and keep the smallest verified result.

Reading The Report

OUTNAME_Report.txt is the best place to check what the compressor decided. Useful lines include:

Report line	Meaning
Max uncompressed packet size	Current -max setting.
Max hash-chain checks	Current -chain or -checks setting.
Compression method	Active -M method.
Zero-fill internal gaps...	Whether -Z was used and how many bytes it added.
Block map	Final physical MMU block usage.
packet file bytes	Size of the compressed stream before LOADM chunk overhead.

The block map legend is:

Marker	Meaning
....	Unused block.
USER	Block touched by the user's program.

LOAD	Loader block.
SHAD	Disk BASIC shadow/protected block.
SRC	Compressed source stream block, if used by that build.

Disk Images And Multi-Disk Builds

CC3_Comp writes DISKFILES.LST with the disk number for each generated COMP####.BIN chunk. The makefile uses that list to create DISK1.DSK, DISK2.DSK, and so on.

DISK1.DSK receives:

```
START.BAS
MOVER.BIN
CC3X0.BIN
COMP####.BIN files assigned to disk 1
```

Later disks receive only their assigned COMP####.BIN files. The BASIC loader uses the decompressor status byte to know when to ask the user for the next disk.

Examples

The example sizes below use the Galaga test build: original NEW.BIN source size 230,950 bytes, with 230,715 bytes of touched program data.

Baseline build:

```
./CC3_Comp NEW.BIN -oGO-V12
```

Typical benchmark result: about 94.8 KB compressed packet data from the 230,950-byte source.

Verbose build:

```
./CC3_Comp NEW.BIN -oGO-V12 -v
```

Fast improved compression:

```
./CC3_Comp NEW.BIN -oGO-V12 -M78 -Z00
```

Typical benchmark result: about 92.7 KB compressed packet data from the 230,950-byte source.

Slower compression test:

```
./CC3_Comp NEW.BIN -oGO-V12 -M716 -Z00
```

Typical benchmark result: about 92.7 KB compressed packet data from the 230,950-byte source, slightly smaller than -M78 but slower to compress.

Quick greedy build with smaller packets and fewer chain checks:

```
./CC3_Comp NEW.BIN -oGO-V12 -q -max4096 -chain128
```

Makefile version:

```
make PROJECT=GALAGA PROGRAM_ASM=Galaga_12.asm
make PROJECT=GALAGA PROGRAM_ASM=Galaga_12.asm COMP_OPTIONS="-M78 -Z00"
```

Troubleshooting

If the tool prints nothing, that is normal. The default mode is quiet. Use -v for progress output.

If a build appears to reuse old options, explicitly clear or set COMP_OPTIONS:

```
make COMP_OPTIONS=  
make COMP_OPTIONS="-M78 -Z00"
```

If the first chunk does not fit on disk 1, try reducing packet size:

```
make COMP_OPTIONS="-max4096"
```

If a program breaks only with -Z, disable -Z and check whether the program depends on bytes it did not explicitly load.

If a program breaks only with an advanced -M method, rebuild with -M0 to confirm the loader path is still healthy, then compare the reports and try a less aggressive method such as -M1 or -M78.